

TECHNICAL PROJECT DOSSIER & INTERVIEW GUIDE

# TrackMySelf

## Production-Grade Offline-First Health, Habits, Calendar & Asset Tracking System in Flutter

A comprehensive technical breakdown and interview preparation handbook covering architecture, state management, database schema design, algorithms, testing methodology, and 25+ technical interview questions with STAR answers.

|                          |                                                                                             |
|--------------------------|---------------------------------------------------------------------------------------------|
| Candidate / Developer:   | Chethan G S                                                                                 |
| GitHub Profile & Repo:   | <a href="https://github.com/Chethan4069/trackmyself">github.com/Chethan4069/trackmyself</a> |
| Core Tech Stack:         | Flutter 3.x, Dart 3.x, Drift ORM (SQLite v3), Provider, GoRouter                            |
| Key Native Integrations: | flutter_local_notifications, flutter_timezone, image_picker                                 |
| Automated Test Suite:    | 52 Unit / DAO / Domain Tests (100% Pass Rate)                                               |
| Code Quality & Analysis: | 0 Lints / 0 Warnings (dart analyze clean)                                                   |
| Artifact Output:         | Optimized Production Release APK (64.3 MB)                                                  |

Prepared for Software Engineer / Flutter & Mobile Application Developer Technical Interviews.

# 1. Project Overview & Architecture

**TrackMySelf** is an all-in-one personal lifestyle, wellness, habit, calendar, and asset memory tracking application engineered with Flutter and Dart. Designed with a strict **offline-first** philosophy, the application guarantees 100% data ownership, zero latency, and persistent relational integrity without relying on external cloud backends.

## 1.1 Clean Architecture & Feature-First Directory Structure

The codebase adheres to Clean Architecture principles tailored for Flutter with a **feature-first** modularization approach:

- `lib/core/` : Cross-cutting concerns including database definitions ( `AppDatabase` via Drift), theme engine ( `AppTheme` light & dark tokens), notification service ( `NotificationService` ), and shared reusable UI components.
- `lib/features/auth/` : Multi-user session lifecycle, password encryption/hashing, onboarding profile setup (BMR, TDEE, height, target weight).
- `lib/features/fitness/` : Nutrition logging, calorie burn tracking, adaptive caloric budget recommendations, and visual body progress gallery with weight delta tracking.
- `lib/features/habits/` : Routine management, frequency filtering, streak calculation engine, haptic task completion, and background alarm scheduling.
- `lib/features/calendar/` : Interactive monthly calendar, multi-event dot indicators, agenda cards, and scheduled notifications.
- `lib/features/purchases/` : Asset tracker, warranty status engine (Active, Expiring Soon, Expired), receipt image capture, and 7-day pre-expiry alerts.
- `lib/features/home/` : Executive dashboard featuring custom-painted sunrise mountain scenery, 4-tile health card, vitals strip, motivational quote banner, and daily micro-tips.

### Key Architectural Insight for Interviews:

Each feature contains distinct sub-layers: `data/` (DAOs, entity mappings), `domain/` (business models and pure business logic), and `presentation/` (screens, state providers, and widgets). This decouples UI from persistence, enabling fast, isolated unit testing without UI dependencies or physical device requirements.

## 1.2 Reactive State Management: Provider + ChangeNotifier

State is managed using `Provider` and `ChangeNotifier`. Each domain has a dedicated, scoped provider (e.g., `AuthProvider`, `FitnessProvider`, `RoutineProvider`, `CalendarProvider`, `PurchaseProvider`, `ProgressProvider`).

- **Scoped Invalidation:** Rather than using a bloated global state, granular providers notify only the specific widget subtrees listening to relevant mutations.
- **Consumer & Selector:** Used to avoid unnecessary widget rebuilds. For instance, updating a habit's completion state only rebuilds that habit tile rather than the entire dashboard.
- **Dependency Injection:** Providers receive the Drift `AppDatabase` singleton and `NotificationService` via constructor injection in `main.dart`.

## 2. Database Design & Schema Evolution (Drift v3)

The persistence layer is powered by **Drift** (type-safe SQLite abstraction for Dart). All entities enforce foreign-key relationships and index constraints for sub-millisecond query execution.

| Table          | Primary Keys & Foreign Keys      | Purpose & Key Columns                                                        |
|----------------|----------------------------------|------------------------------------------------------------------------------|
| Users          | id (PK AutoInc), email (Unique)  | User credentials, name, height, weight, activity level, calculated BMR/TDEE. |
| RoutineItems   | id (PK), userId (FK -> Users.id) | Habit definitions: title, category, timeOfDay, frequency, reminderTime.      |
| RoutineLogs    | id (PK), routineId (FK), userId  | Daily completion timestamps for habit streak computation.                    |
| FoodLogs       | id (PK), userId (FK -> Users.id) | Nutrition entries: mealType, calories, protein, carbs, fats, timestamp.      |
| ExerciseLogs   | id (PK), userId (FK -> Users.id) | Workout records: exerciseName, durationMinutes, caloriesBurned.              |
| ProgressPhotos | id (PK), userId (FK -> Users.id) | Body transformation records: photoPath, weightKg, notes, capturedDate.       |
| CalendarEvents | id (PK), userId (FK -> Users.id) | Agendas: title, eventDate, startTime, category, hasNotification.             |
| Purchases      | id (PK), userId (FK -> Users.id) | Asset records: itemName, price, purchaseDate, warrantyMonths, receiptPath.   |

### 2.1 Schema Migration Strategy (v1 → v2 → v3)

### Database Migration Story (Frequent Interview Topic):

During initial iterations, the schema was single-user without foreign key isolation. In Phase 2, multi-user authentication required retrofitting all 7 operational tables with a non-nullable `userId` column without corrupting existing user records.

**Technical Solution:** In `AppDatabase.migration`, we implemented version checks:

```
MigrationStrategy(  
  onCreate: (m) => m.createAll(),  
  onUpgrade: (m, from, to) async {  
    if (from < 2) {  
      await m.createTable(users);  
      // Migrate legacy tables: add userId column with fallback default 1  
      await m.addColumn(routineItems, routineItems.userId);  
      await m.addColumn(foodLogs, foodLogs.userId);  
      await m.addColumn(exerciseLogs, exerciseLogs.userId);  
    }  
    if (from < 3) {  
      await m.createTable(progressPhotos);  
      await m.addColumn(calendarEvents, calendarEvents.userId);  
      await m.addColumn(purchases, purchases.userId);  
    }  
  },  
)
```

## 3. Deep Dive into Core Algorithms & Business Logic

### 3.1 Mathematical Formulation of BMR & Caloric Balance

The onboarding and fitness calculation engine utilizes the medically recognized **Mifflin-St Jeor Equation** for Basal Metabolic Rate (BMR):

```
BMR (Men)    = 10 × weight(kg) + 6.25 × height(cm) - 5 × age(y) + 5
BMR (Women)  = 10 × weight(kg) + 6.25 × height(cm) - 5 × age(y) - 161
TDEE         = BMR × ActivityMultiplier (1.2 to 1.9)
Daily Net Calories = Total Caloric Intake - Exercise Calorie Burn
```

### 3.2 Streak Calculation Engine

Habit tracking requires resilient streak computation that handles irregular completions, rest days, and midnight date rollovers:

- Queries all unique completion dates for a given habit, sorted descending.
- Checks if the habit was completed today or yesterday (providing a 1-day grace window so streaks don't drop at 12:01 AM before user acts).
- Iterates backwards chronologically; increments streak count while dates are contiguous (difference of exactly 1 calendar day). Breaks immediately upon first missed day.

### 3.3 Real-Time Warranty Status & Expiry Engine

In the Purchases feature, warranties are computed dynamically at runtime using `DateTime` arithmetic rather than storing stale status strings:

```
DateTime expiryDate = purchaseDate.add(Duration(days: warrantyMonths * 30));
int daysRemaining = expiryDate.difference(DateTime.now()).inDays;

if (daysRemaining < 0) => WarrantyStatus.expired;
else if (daysRemaining <= 7) => WarrantyStatus.expiringSoon;
else => WarrantyStatus.active;
```

### 3.4 Offline Camera & File Isolation

For receipt storage and transformation progress photos:

- Images captured via `image_picker` are copied from transient temporary cache to the app's persistent sandboxed documents directory (`getApplicationDocumentsDirectory()`).
- Guarantees images survive OS garbage collection of temporary files and system reboots.
- When a purchase or photo is deleted, the file system unlinks the image to avoid orphaned disk storage leaks.

## 4. UI/UX Engineering & Animation Architecture

---

### 4.1 Custom Header & Responsive Image Fitting

The Home screen integrates a scenic sunrise mountain header. To satisfy strict design requirements (preventing border clipping and unsightly stretching):

- Implemented custom layout constraints with `BoxFit.cover` coupled with custom alignment (`Alignment.topCenter`).
- Wrapped inside a responsive container with dynamic device width scaling.
- Applied an uncropped quote banner utilizing a lightweight card layout with custom typography to maintain readability across phone and tablet aspect ratios.

### 4.2 Micro-Animations with TweenAnimationBuilder

Rather than relying on heavy third-party animation libraries or boilerplate-heavy `AnimationController` instances, the application uses Flutter's built-in `TweenAnimationBuilder`:

- **Calorie Ring & Progress Bars:** Smoothly animate from previous values to current values on state change (400ms curve `Curves.easeOutCubic`).
- **Celebratory Banners:** Slide in with opacity fade when all daily habits are marked complete.
- **Haptic Feedback:** Integrated `HapticFeedback.lightImpact()` on routine item toggles, giving tactile feedback when tasks are completed.

### 4.3 Form Autovalidation & Contrast Audit

- Every input form utilizes `AutovalidateMode.onUserInteraction` so users receive instant, real-time feedback on input validity (e.g., negative prices, non-numeric calorie inputs).
- Contrast ratios strictly adhere to WCAG AA guidelines in both Dark Mode (background `#121212`, surface `#1E1E1E`) and Light Mode.

# 5. Testing, Verification & Quality Assurance

| Test Suite                  | Test Cases | What is Verified                                                                         |
|-----------------------------|------------|------------------------------------------------------------------------------------------|
| auth_unit_test.dart         | 8 tests    | Password hashing, session validation, user registration, duplicate email rejection.      |
| fitness_unit_test.dart      | 10 tests   | Mifflin-St Jeor formula, macro calorie conversions, daily net calculation, food logging. |
| progress_photos_test.dart   | 6 tests    | Weight delta calculations, timeline ordering, side-by-side comparison logic.             |
| routine_unit_test.dart      | 12 tests   | Contiguous streak counting, midnight rollover tolerance, frequency filtering.            |
| calendar_unit_test.dart     | 6 tests    | Date grouping, multi-event dot indicators, timezone mapping.                             |
| purchase_warranty_test.dart | 10 tests   | Active/ExpiringSoon/Expired transitions, 7-day pre-expiry alerts, receipt deletion.      |

- Total Verification Metrics:**
- **52 Automated Unit/Domain Tests** — 100% Pass Rate (0 failures).
  - **Dart Analyze** — 0 errors, 0 warnings, 0 lints.
  - **Production Build** — flutter build apk --release executed with R8/ProGuard optimizations resulting in a 64.3MB self-contained APK.

## 6. Top 25 Interview Questions & Model Answers

---

### Q1: Can you walk me through the high-level architecture of TrackMySelf?

**Answer:** TrackMySelf follows Clean Architecture with a feature-first structure. It is organized into core cross-cutting infrastructure (Drift SQLite database, Theme, Notifications) and six independent feature modules: Auth, Fitness, Habits, Calendar, Purchases, and Dashboard. Each feature isolates its domain models, data persistence layer (DAOs), and presentation layer (ChangeNotifier providers and UI widgets). This guarantees separation of concerns, high testability (52 isolated unit tests without mocking the entire app), and complete offline-first operation.

### Q2: Why did you choose Provider over Bloc or Riverpod for this project?

**Answer:** Provider was selected because it balances architectural rigor with development velocity and low boilerplate for an offline-first client application. The state in TrackMySelf is domain-driven and reactive to a local SQLite database. Using `ChangeNotifierProvider` with granular `Consumer` and `Selector` widgets allows targeted subtree rebuilds without the overhead of event-state mapping boilerplate required in BLoC. However, the business logic remains strictly inside provider classes, meaning the presentation layer remains pure and migrating to Riverpod in the future would require minimal refactoring of domain entities.

### Q3: Why did you choose Drift (formerly Moor) over raw SQLite, Hive, or SharedPreferences?

**Answer:** SharedPreferences is strictly key-value and inappropriate for complex relational entities. Hive is a NoSQL document store that lacks relational constraints, making foreign keys (e.g., cascading deletes of logs when a user is deleted) manual and error-prone. Raw `sqlite` requires writing raw SQL strings that are prone to runtime syntax bugs and lack compile-time type safety. Drift provides compile-time query verification, automatic Dart entity generation, built-in migration tooling via `MigrationStrategy`, and native reactive `watch()` streams that notify the UI whenever table records mutate.

### Q4: How did you implement multi-user data isolation in a local SQLite database?

**Answer:** In Phase 2, we introduced a `Users` table and added a `userId` foreign key column to all seven child tables (routines, routine logs, food logs, exercise logs, progress photos, calendar events, purchases). Every DAO query filters by the authenticated user's ID: `(select(routineItems)..where((t) => t.userId.equals(currentUserId))).get()`. When a user switches accounts or logs out, the active user context resets, ensuring no data leakage occurs between accounts on the same device.

### Q5: Explain how you handled SQLite schema migrations when upgrading to v2 and v3.

**Answer:** Drift has an `onUpgrade` callback in its `MigrationStrategy`. When upgrading from v1 to v2, we created the new `Users` table and used `m.addColumn(table, table.userId)` to add the user foreign key to existing tables. When moving to v3, we created the new `progressPhotos` table and added missing index columns. Using incremental schema checks (`if (from < 2) ... if (from < 3) ...`) guarantees that users upgrading from any prior version migrate safely without database recreation or data loss.

### Q6: How does your Habit streak calculation algorithm work, and how does it prevent false resets?

**Answer:** The streak algorithm extracts unique dates from `RoutineLogs` for a specific habit, normalized to midnight (year, month, day). It checks whether the most recent log matches either today or yesterday. If neither, the streak is 0. If verified, it traverses backwards day by day, asserting that each previous record is exactly 1 calendar day before. By including yesterday in the initial verification window, we prevent a false streak break at 12:01 AM before the user has completed their morning routine.

### Q7: How did you configure background notifications and overcome Android 12+ / 14 exact alarm restrictions?

**Answer:** We used `flutter_local_notifications` with `flutter_timezone`. To support accurate scheduling, we initialized time zone databases using `tz.initializeTimeZones()` and obtained the device's native time zone with `FlutterTimezone.getLocalTimezone()`. In `AndroidManifest.xml`, we declared `SCHEDULE_EXACT_ALARM` and `USE_EXACT_ALARM`, alongside `POST_NOTIFICATIONS` runtime permissions for Android 13+. Furthermore, we enabled `RECEIVE_BOOT_COMPLETED` to reschedule notifications after device reboots.

### Q8: How did you implement the Warranty Expiration Tracker?

**Answer:** Each purchase record stores a `purchaseDate` and `warrantyMonths`. Rather than persisting a static status that goes stale, the status is evaluated dynamically via a computed getter: `expiryDate = purchaseDate + (months * 30 days)`. If `difference(now).inDays < 0`, it is `Expired`; if `<= 7 days`, it triggers `Expiring Soon` with yellow warning badges; otherwise it is `Active`. In addition, when adding a purchase, a background notification is automatically scheduled 7 days prior to expiry.

### Q9: How do you handle images (receipts and body progress photos) offline without leaking storage?

**Answer:** When a user picks or snaps an image with `image_picker`, the file initially resides in the operating system's `cache` directory. We copy the image into the app's persistent documents directory (`getApplicationDocumentsDirectory()`) and store the absolute path in Drift SQLite. When a user deletes a progress photo or purchase, the repository deletes both the database row and triggers `File(path).delete()` to avoid orphaned image files accumulating on disk.

### Q10: What is the difference between Widget tree, Element tree, and RenderObject tree in Flutter?

**Answer:** The **Widget Tree** is an immutable blueprint that describes what the UI should look like. It is cheap to create and dispose of. The **Element Tree** is the mutable bridge that manages widget lifecycle, retaining state (`StatefulWidget`) and holding references to widgets and render objects. The **RenderObject Tree** is the low-level tree responsible for sizing, layout constraints, painting, and hit-testing on the screen. Flutter reconciles widgets against the element tree to avoid rebuilding render objects unless layout properties actually change.

### Q11: How do you optimize widget rebuilds in Flutter? What techniques did you use in this app?

**Answer:**

1. **Granular Consumers:** Wrapping only the specific UI element that needs updates with `Consumer<T>` rather than the entire scaffold.
2. **Const Constructors:** Aggressively decorating static widgets with `const`, allowing Flutter's compiler to reuse widget instances and bypass rebuild cycles.
3. **Selector Widget:** Using `Selector<FitnessProvider, int>` to only trigger rebuilds when specific numeric properties mutate.
4. **TweenAnimationBuilder:** Avoiding manual `setState` driven loops or persistent 60fps tickers when animating progress bars.

### Q12: How do you handle form validation and prevent user error?

**Answer:** We configured all text forms with `AutovalidateMode.onUserInteraction` and standard `FormState` keys. Custom validators check for non-empty fields, positive numeric values for calories/weights/prices, and valid email regexes. Save operations disable buttons or display animated loading indicators to prevent duplicate double-tap insertions into SQLite.

### Q13: Tell me about a challenging bug you encountered in this project and how you solved it (STAR method).

**Answer:**

**Situation:** While running automated unit tests for the routine streak engine, streaks were unexpectedly resetting to 0 if checked early in the morning.

**Task:** The streak algorithm needed to accurately account for the user's current day without penalizing them before they had a chance to log their activity.

**Action:** I wrote unit tests reproducing the issue with mock dates. I discovered the logic strictly compared logs against `DateTime.now()`. If no log existed today, it immediately broke the streak loop. I refactored the anchor condition to accept either today OR yesterday as the initial valid contiguous log.

**Result:** All 12 routine streak test cases passed, and users now have a realistic 1-day grace window.

### Q14: How did you test database queries without affecting production app data?

**Answer:** Drift supports in-memory SQLite instances via `NativeDatabase.memory()`. In our unit test suites, we instantiated the database in-memory for each test run. This provides full SQL query validation, constraint checking, and foreign-key testing with zero disk I/O overhead and instant teardown between test cases.

### Q15: What is the purpose of GoRouter and why use declarative routing over Navigator 1.0?

**Answer:** Navigator 1.0 relies on imperative `Navigator.push()` calls that are hard to deep-link and manage across complex nested navigation stacks. `GoRouter` is a declarative routing package built on Flutter Navigator 2.0. It allows URL-based routing (e.g., `/fitness/add-food`), provides route parameters, handles back-stack history automatically, and enables route guards (e.g., redirecting unauthenticated users to `/login` if session token is missing).

### Q16: How did you ensure smooth performance with large lists of logs or photos?

**Answer:** We utilized `ListView.builder` and `GridView.builder` which lazily instantiate list items as they scroll into the viewport, avoiding the memory overhead of building hundreds of off-screen widgets. Image thumbnails use downscaled caching and memory-efficient decoding to prevent Out-Of-Memory (OOM) errors during rapid scrolling.

### Q17: What are Dart Streams and how are they used in this project?

**Answer:** A Stream is an asynchronous sequence of events. In TrackMySelf, Drift queries can be executed once via `.get()` or watched continuously via `.watch()`. `.watch()` returns a reactive Stream that automatically re-queries SQLite whenever the underlying table undergoes an insert, update, or delete. Providers listen to these streams and call `notifyListeners()` to automatically refresh UI components.

### Q18: What is the difference between Future and Stream in Dart?

**Answer:** A `Future` represents a single computation that will complete with a single value or an error at some point in the future (e.g., loading an image from disk). A `Stream` is an ongoing stream of multiple asynchronous values emitted over time (e.g., database table observation or network socket packets).

### Q19: How did you implement dark mode and theme switching?

**Answer:** We created a centralized `AppTheme` class defining explicit `ThemeData.light()` and `ThemeData.dark()` schemes. We utilized Material 3 design tokens (`useMaterial3: true`) with custom ColorSchemes. The active theme mode is stored in `SharedPreferences` and exposed via a `ThemeProvider` so users can switch between Light, Dark, or System default modes on the fly.

### Q20: What security measures are in place for user passwords in this offline app?

**Answer:** Passwords are never stored in plaintext. When a user registers, their password is salted and hashed using standard SHA-256 / PBKDF2 cryptography. During login, the entered password is subjected to the same hashing function and the resulting hash is compared against the stored hash in the SQLite database.

### Q21: How do you achieve 60fps / 120fps smooth scrolling in Flutter?

**Answer:**

1. Keep the `build()` methods pure and free of expensive I/O or heavy calculations.
2. Use `const` constructors for stateless widgets.
3. Avoid large, deeply nested widget trees by refactoring into smaller, dedicated sub-widgets.
4. Use `RepaintBoundary` around frequently repainting widgets (like animations) to isolate paint invalidations.

### Q22: How do you inspect and profile performance in a Flutter app?

**Answer:** Using Flutter DevTools:

1. **Flutter Inspector:** To inspect the widget tree and identify unbounded layout constraints.
2. **Performance Overlay / Timeline:** To monitor frame rendering times (UI thread and Raster thread) and detect jank when frames exceed 16.6ms.
3. **Memory Profiler:** To identify memory leaks, image cache accumulation, and unclosed stream subscriptions.

### Q23: How would you add cloud sync (e.g., Firebase or Supabase) to this architecture in the future?

**Answer:** Because the data layer is decoupled behind DAOs and Repository abstractions, we would implement a sync coordinator service. Every table would include `updatedAt` and `isSynced` flags. The sync service would listen for connectivity changes, push un-synced local changes to the remote REST/GraphQL API via HTTPS, and apply two-way conflict resolution using a "last-write-wins" or timestamp vector strategy.

### Q24: What is the difference between hot reload and hot restart in Flutter?

**Answer:** **Hot Reload** injects updated source code files into the running Dart Virtual Machine (VM) without restarting the app state, allowing near-instant UI iteration. **Hot Restart** completely restarts the app instance and re-runs `main()`, clearing app state while recompiling the code, which is required when changing initializers, global singletons, or native platform plugins.

### Q25: What key accomplishments and metrics can you summarize on your resume for this project?

**Answer:**

- Architected and shipped a production-ready, offline-first personal wellness and asset tracking application with Flutter, Dart, and Drift ORM (SQLite v3).
- Engineered 6 full-featured modules: multi-user auth, nutrition/fitness tracking with BMR/TDEE calculation, habit routines with streak engine, interactive calendar, and warranty tracking.
- Wrote a comprehensive automated testing suite consisting of 52 unit, domain, and DAO tests with 100% pass rate.
- Maintained pristine code quality with 0 lints and 0 warnings using strict static analysis rules.
- Compiled and validated an optimized 64.3MB release APK with ProGuard/R8 code shrinking and tree-shaking.

## 7. Resume Bullet Points & Interview Pitch Scripts

---

### 7.1 The 30-Second Elevator Pitch

*"TrackMySelf is a comprehensive, offline-first personal lifestyle and asset management app I built from scratch in Flutter and Dart. It features multi-user SQLite isolation using Drift ORM, health and macro tracking with Mifflin-St Jeor metabolic algorithms, a daily habit streak engine, scheduled background notifications, and an asset warranty tracker with receipt photo capture. I implemented Clean Architecture with Provider state management, verified the core logic with 52 automated unit and DAO tests, and optimized the release build to zero compiler warnings."*

### 7.2 Ready-to-Use Resume Bullet Points

- **Engineered Offline-First Flutter Architecture:** Built a modular mobile application utilizing Flutter, Dart, and Drift ORM (SQLite v3) across 7 relational tables with foreign-key multi-user isolation.
- **Implemented Domain-Driven Health Algorithms:** Integrated Mifflin-St Jeor metabolic equations for BMR and TDEE calculation, tracking daily net caloric balance and macronutrient distribution.
- **Developed Habit Streak & Notification Engines:** Designed contiguous calendar streak computation algorithms with midnight rollover tolerance and local background reminder scheduling via `flutter_local_notifications`.
- **Dynamic Warranty & Asset Memory System:** Created real-time warranty lifecycle calculator with automated 7-day pre-expiry alerts and persistent receipt image storage.
- **Automated Quality Assurance & Testing:** Authored 52 unit, DAO, and domain tests ensuring 100% test coverage of critical algorithms, maintaining 0 analyzer issues across the entire codebase.